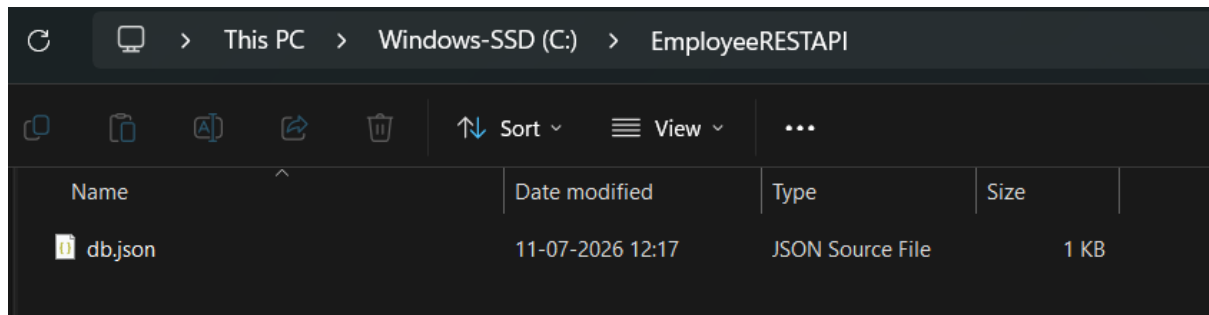


# REST API Mini Project – Employee Management System

## Step 1: Project Folder Creation

EmployeeRESTAPI folder containing `db.json`



### Purpose

To create the project workspace where the employee data and REST API configuration files are stored.

### Description

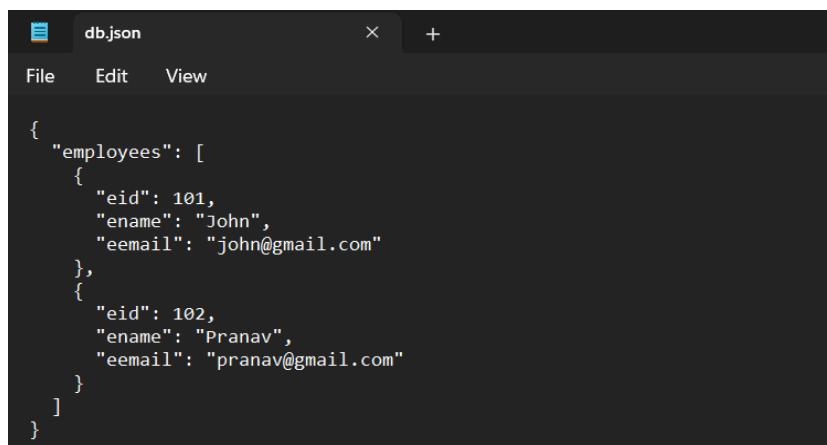
A project folder named **EmployeeRESTAPI** was created. Inside this folder, the `db.json` file was added, which acts as the database for JSON Server.

### Result

The project structure was successfully created.

## Step 2: Creating the Database (db.json)

`db.json` file opened in the editor.



## Purpose

To create a fake database containing employee information.

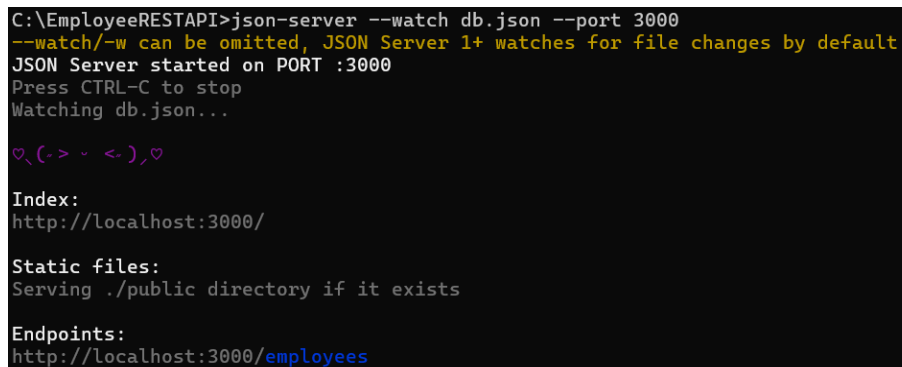
## Description

The `db.json` file was created with an `employees` array containing employee details such as Employee ID, Employee Name, and Email Address. This file is used by JSON Server to generate REST API endpoints automatically.

## Result

Employee records were successfully stored in JSON format.

## Step 3: Starting JSON Server

A terminal window showing the command to start JSON Server. The command is `C:\EmployeeRESTAPI>json-server --watch db.json --port 3000`. The output shows that the server is watching for file changes and has started on port 3000. It also displays the index URL, static files directory, and the endpoints for the REST API.

```
C:\EmployeeRESTAPI>json-server --watch db.json --port 3000
--watch/-w can be omitted, JSON Server 1+ watches for file changes by default
JSON Server started on PORT :3000
Press CTRL-C to stop
Watching db.json...

♥\(-> ~ <-)♥

Index:
http://localhost:3000/

Static files:
Serving ./public directory if it exists

Endpoints:
http://localhost:3000/employees
```

## Purpose

To start the Fake REST API server.

## Description

**The JSON Server was started using the command:**

- `json-server --watch db.json --port 3000`

**The server started successfully on Port 3000 and generated the REST endpoint:**

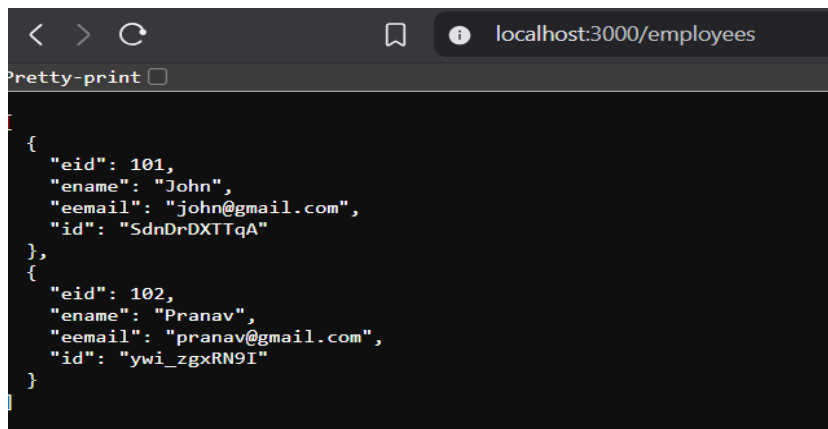
- `http://localhost:3000/employees`

## Result

The REST API server started successfully.

## Step 4: Verifying API in Browser

### Browser displaying



<http://localhost:3000/employees>

### Purpose

To verify whether the REST API is running correctly.

### Description

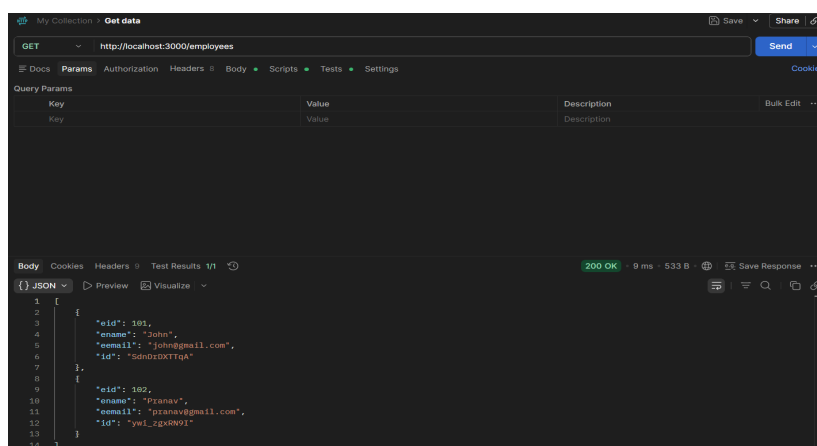
The generated endpoint was opened in the browser. The employee records stored inside `db.json` were displayed in JSON format.

### Result

The API endpoint was successfully verified.

## Step 5: GET Request – Retrieve All Employees

### Postman GET



<http://localhost:3000/employees>

## Purpose

To retrieve all employee records.

## Description

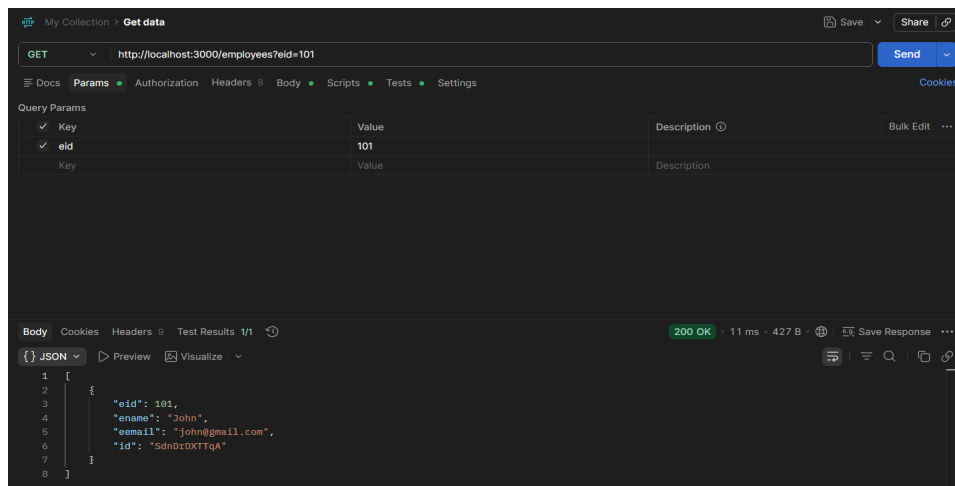
A GET request was sent using Postman. The server returned all employee records available in the database.

## Result

Employee details were retrieved successfully with **200 OK** status.

## Step 6: GET Request – Search Employee by Employee ID

### GET



- `employees?eid=101`

## Purpose

To search an employee using the Employee ID.

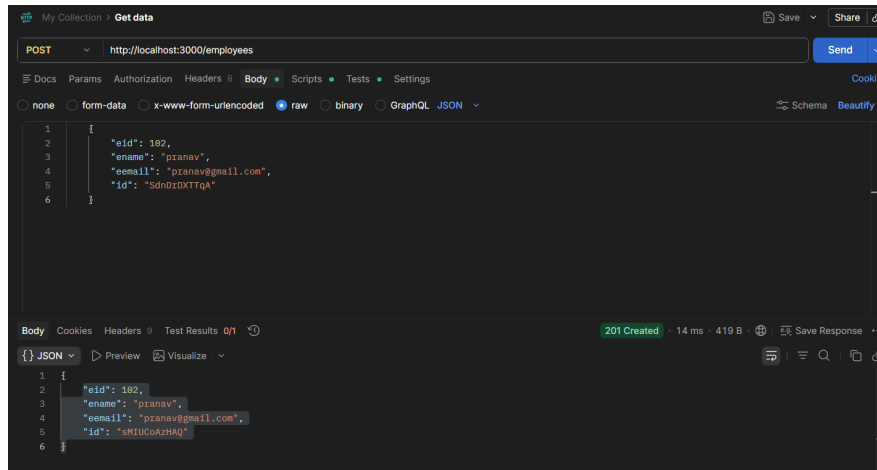
## Description

A GET request was executed using the query parameter `eid=101`. JSON Server filtered the records and returned the employee whose Employee ID is 101.

## Result

The required employee details were displayed successfully.

## Step 7: POST Request – Add New Employee



POST request with employee details.

### Purpose

To insert a new employee into the database.

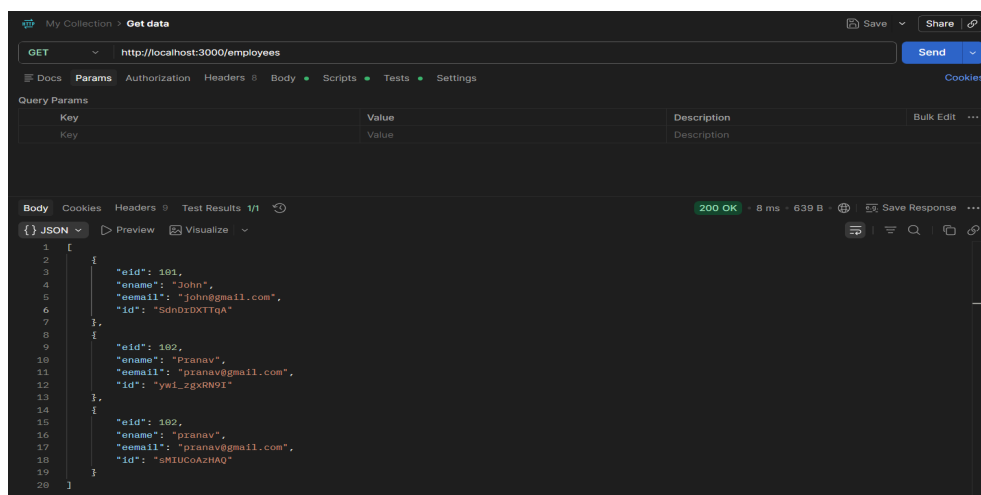
### Description

A POST request was sent to the `/employees` endpoint along with employee details in JSON format. JSON Server created a new employee record and generated a unique `id`.

### Result

A new employee record was successfully created with **201 Created** status.

## Step 8: Verify Newly Added Employee



GET request showing three employees.

## Purpose

To verify that the new employee has been added successfully.

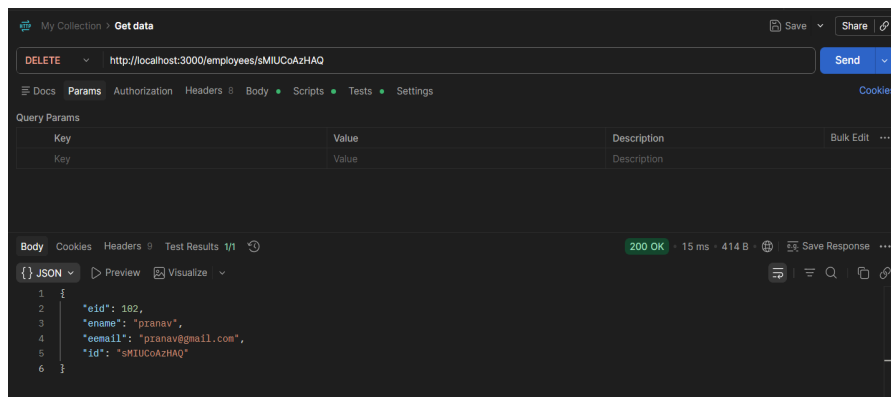
## Description

After performing the POST request, another GET request was executed. The response displayed all employees, including the newly inserted employee.

## Result

The new employee was successfully added to the database.

## Step 9: DELETE Request



## DELETE request

## Purpose

To remove an employee record from the database.

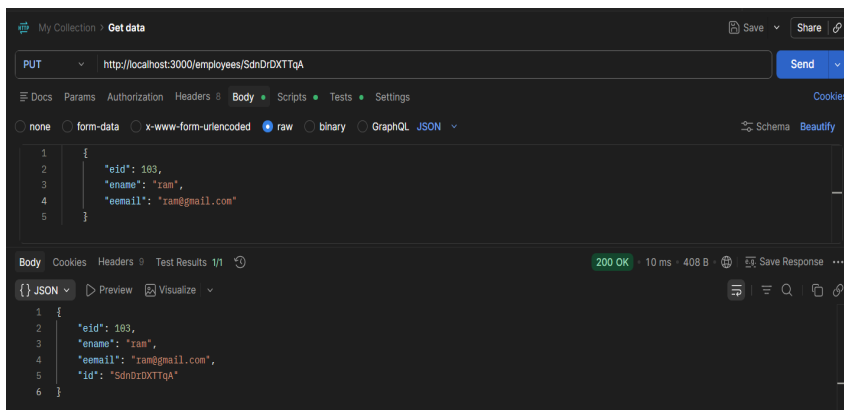
## Description

A DELETE request was sent to the employee endpoint using the employee's unique ID. JSON Server deleted the corresponding employee record from the database.

## Result

The employee record was deleted successfully.

## Step 10: PUT Request – Update Employee



### PUT request

#### Purpose

To replace the existing employee information.

#### Description

A PUT request was sent with updated employee details. The employee's information was completely replaced with the new values.

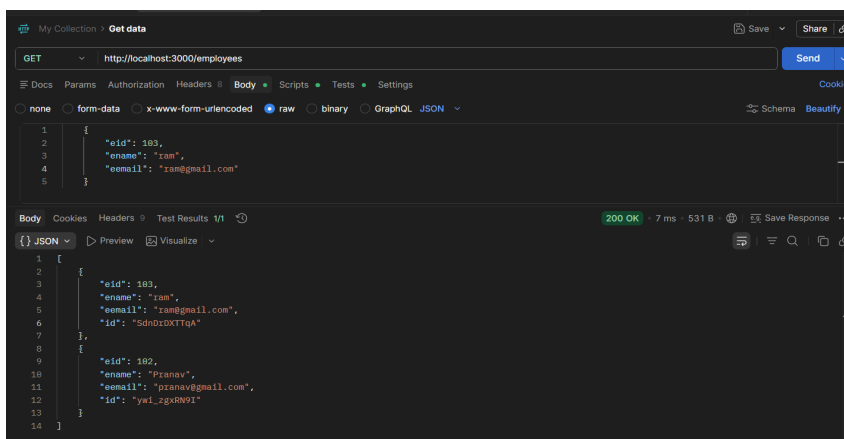
Updated Details:

- Employee ID: 103
- Name: Ram
- Email: ram@gmail.com

#### Result

The employee information was updated successfully.

## Step 11: Verify Updated Employee



## GET request after PUT

### Purpose

To confirm that the update operation was successful.

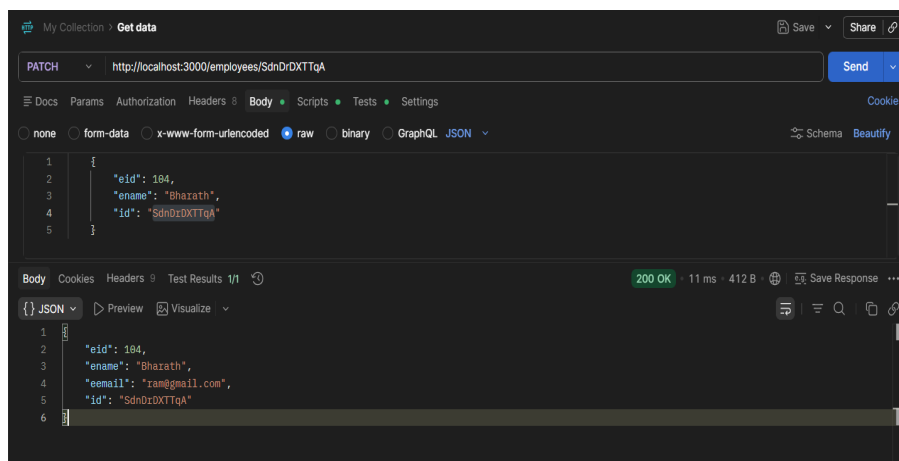
### Description

A GET request was performed after the PUT operation. The response displayed the updated employee information.

### Result

The updated employee details were retrieved successfully.

## Step 12: PATCH Request – Partial Update



### PATCH request

#### Purpose

To update only selected fields of an employee record.

#### Description

A PATCH request was executed to modify only the Employee ID and Employee Name while keeping the Email Address unchanged.

Updated Details:

- Employee ID: 104
- Name: Bharath

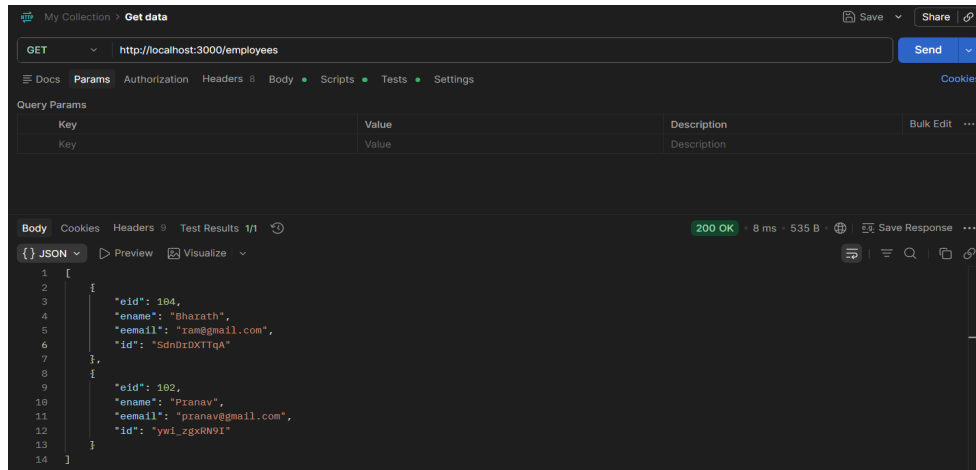
The Email Address remained unchanged.



## Result

The selected employee fields were updated successfully.

## Step 13: Verify PATCH Operation



## GET request after PATCH

### Purpose

To verify that only the required fields were modified.

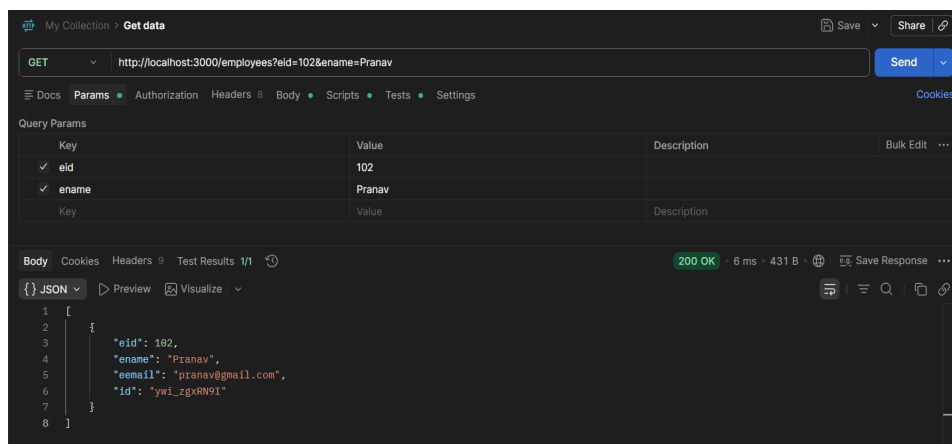
### Description

A GET request was sent after the PATCH operation. The response confirmed that the Employee ID and Name were updated while the Email Address remained the same.

## Result

The PATCH operation was verified successfully.

## Step 14: Search Employee Using Multiple Query Parameters



## **GET**

- `employees?eid=102&ename=Pranav`

## **Purpose**

To search an employee using multiple conditions.

## **Description**

A GET request was executed using two query parameters: Employee ID and Employee Name. JSON Server returned only the employee matching both conditions.

## **Result**

The employee record satisfying both query parameters was retrieved successfully.

## **Conclusion**

The Employee Management System REST API was successfully implemented using JSON Server. The `db.json` file acted as the database, and REST API endpoints were tested using Postman. All CRUD operations—Create (POST), Read (GET), Update (PUT/PATCH), and Delete (DELETE)—were executed successfully. The project demonstrates the practical implementation of RESTful web services for managing employee records efficiently.

